

Wstępny pomysł: Szpital

scenariusz:

Sala pooperacyjna, obecne systemy (jeżeli w ogóle są) zakładają binarność zachowań: pacjent leży, pacjent wstał. Nawet gdyby wstawić tam klasyfikacje taka jak yolo, moglibyśmy określić czy jest w pozycji siedzącej, lub czy to jest człowiek czy krzesło przy dobrej klasyfikacji moglibyśmy sklasyfikować i nazwać konkretny ruch. Nasze rozwiązanie mogłoby symulować całodobową opiekę opisując co widzi na bieżąco w sali czy pacjent jest chwiejny, czy spada z łóżka, czy próbuje odpiąć kabel, czy próbuje coś zrobić dziwnego. Problem: model sam w sobie jest fotoreporterem zwraca opis tego co widzi w kilku krokach, można powiedzieć że opis jest dokładny pod względem kontekstu, składni oraz analizy sytuacji, natomiast osoba siedząca na skraju łóżka jest osobą siedzącą na skraju łóżka a nie osobą która potencjalnie zemdląca, pierwszy plan zakładałam fine tuning - bardzo kosztowny obliczeniowo, na pewno by nie wyszło, na szczęście modele VLM mają w sobie LLM, które są bardzo podatne na prompt engineering, co uważamy że może być dobrym rozwiązaniem, możemy powiedzieć modelowi kim jest jakie ma cechy, jakie ma priorytety i na co zwracać uwagę, przez to jesteśmy w stanie zmienić fotoreportera w model który posiada jakiegokolwiek poziom awarness.

Jakby to mogło wyglądać:

Przychodzi klatka z kamery do systemu zostaje opisana, opis w jakim ustalony sposób byłby zapisywany na systemie aby móc analizować kilka klatek w przód lub w tył żeby złapać kontekst, następnie na ekran użytkownika zostaje wyświetlone czy coś wymaga uwagi czy wszystko wydaje się w porządku.

poc:

1. Awarness przez prompting: model będzie wiedział jakie ma zadanie
2. Temporal context: System zapamiętuje co działo się minute temu, system wie że pacjent siedział dłuższą chwilę a nie dopiero co usiadł
3. Filtrowanie informacji: Na ekran wyrzucane jest Sygnal a nie surowy tekst

Architektura:

1. Kamera
2. Głowa - VLM + Prompt engineering
3. Memory - Context Buffer
4. Dashboard - UI np. streamlit

problem:

klasyfikatory widzą obiekty, a problemem jest rozpoznać sytuację kontekst.

rozwiązanie:

hybrid ai (VLM) - AI Vision + Language: używam LLM wbudowanego w VLM, aby zinterpretować obrazy w kontekście procedur medycznych

skalowalność:

history_log zmieniliby się w bazę danych a file uploader w strumień z kamer.

demo:

symulacja sekwencji zdarzeń: Klatka 1 Niepokój-> Klatka 2 Upewnienie się -> Klatka 3 Działanie

dlaczego jest ok:

naturalna zdolność modeli językowych do rozumienia instrukcji, nie trzeba uczyć modelu czym jest upadek na milionowym datasetcie

na plakacie można napisać "System analizuje trend zachowań w czasie rzeczywistym"

Jest skalowalny - jeżeli ten projekt dostałby duże finansowanie to mógłby zostać użyty

Nie wymaga chmurowej infrastruktury, wystarczy komputer w sali kółka a efekt jest wizualny i łatwy do przedstawienia

Uogólnienie:

Context-Aware Visual Monitoring System

1. Świadomość roli (prompt engineering)
2. Analiza temporalna (memory buffer)
3. Sygnał: wynikiem nie jest surowy tekst tylko alert decyzyjny

Use case:

1. Szpital opisany wyżej
2. Bezpieczeństwo publiczne - to samo co szpital tylko analiza podejrzanych zachowań, obserwowanie strefy chronionej, przemoc lub agresja.
3. Przemysł i bezpieczeństwo pracy - Wykrywanie nie tylko braku kasu ale niewłaściwego użycia narzędzi lub pracy w niebezpiecznej pozycji. Monitorin stref informacja gdy pracownik przebywa w strefie niebezpiecznej dłużej niż dopuszczalny czas

System jest analitykiem sytuacji, posiada zdolności rozumienia LLM, oraz zdolności widzenia, przez co jest w stanie rozróżniać "siedzenie" od "siedzenia z zamiarem wstania" i generować alerty oparte na kontekście a nie tylko geometrii obrazu tak jak klasyfikatory

Opis ogólny założenia systemu

Dokładniejszy opis głównego "docelowego użytkownika"

2 mniejsze use case bardziej rzucone

Scenariusz:

System który dostaje na wejściu klatki -> analiza sytuacji -> podjęcie decyzji

Wchodzi 3 klatki do kamery, A,B,C

Klatka A - dwójka ludzi stojących obok siebie

Klatka B - jeden człowiek atakuje drugiego - ALERT bójka

Klatka C - jeden człowiek leży drugi ucieka - ALERT poszkodowany

Opis klatki A - Widzę dwójkę mężczyzn którzy stoją obok siebie - najważniejsze 2 osoby, obok siebie

Opis klatki B - (pamiętam że dwóch mężczyzn stało obok siebie) - Jeden mężczyzna atakuje drugiego

Opis klatki C - (pamiętamy że dwóch mężczyzn stało obok siebie i jeden zaatakował drugiego) - Jedna z osób ucieka a druga leży na ziemi.

Alert - bójka - poszkodowany - wysłać kartkę

z systemem: uratowany człowiek, bez - człowiek umiera

Stack: VLM (moondream2), wymyślić sposób jak przechowywać takie elementy o których musi pamiętać i wklejać to do nowych promptów.

np. tak:

```
{System prompt}
```

```
{Prompt}
```

```
Last X frames - .....
```

```
Rest of the prompt
```

Docelowo baza danych?

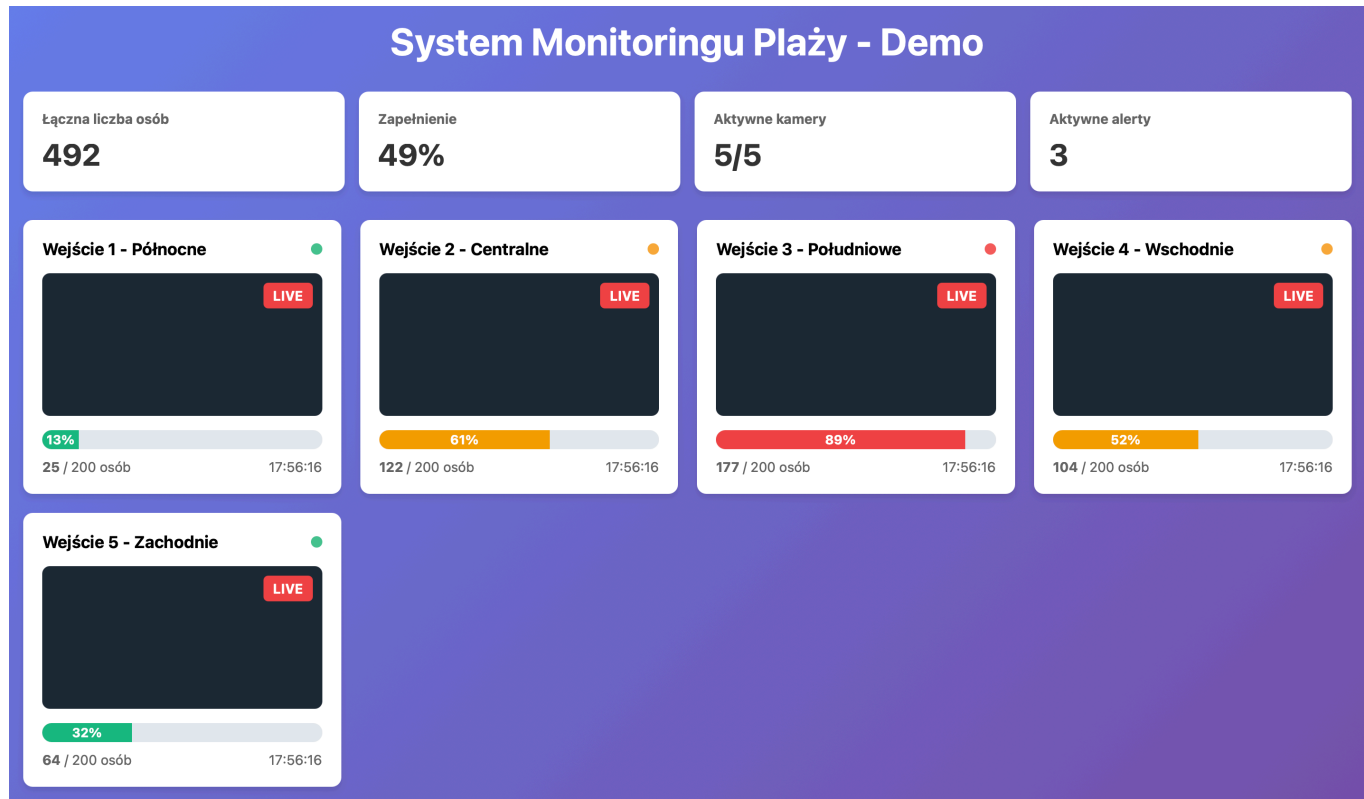
Ja to widzę tak:

1. OpenCV: Wchodzi nagranie
2. Preprocessing: Podział na klatki / kompresja - optymalizacja
3. GPU Server (moondream2): Query do klatki, analiza
4. Python: Program do zapamiętywania ostatnich analiz, jakies deque czy cos
5. Alert system: FastAPI np.

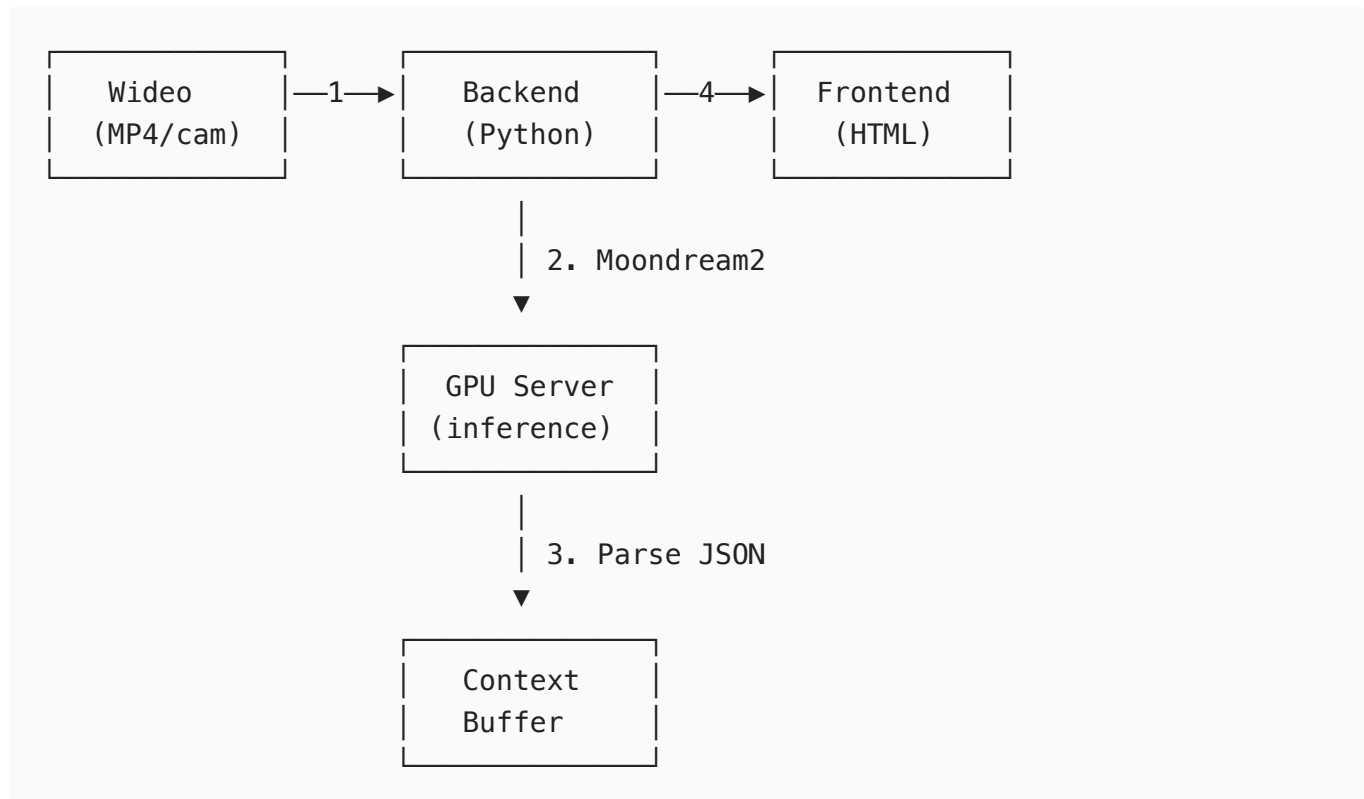
6. Front end: React

Mozna zrobic przyklad z plażą, mamy 5 kamer, każda odpowiada za analize zatłoczenia.

Przykładowy Front end



Przykładowe workflow



Jak to by mogło wyglądać:

Mamy backend na którym jest załadowany model, przychodzi do niego np. Klatka 1 z kamery 1, wysyłamy query do modelu z zapytaniem jak bardzo uważa że ta klatka jest zatłoczona oraz żeby plus minus oszacować liczbę osób na obrazku - sprawdzić jak dokładnie to robi

Następnie dostajemy zwrot w postaci - bardzo zatłoczone około 80 osób, to łąduje w context Buffer, gdybysmy potrzebowali tego w przyszłości oraz żeby wychwycić potencjalne błędy modelu, jeżeli jest - wypełniona- wypełniona - pusta - wypełniona - wypełniona w przeciągu około 10 sekund no to najpewniej informacja 'pusta' to błąd systemu. Następnie odpowiednio łączymy backend z frontem i wyświetlamy te analizy na głównej stronie.

Gdzie można tego użyć? Wszędzie gdzie chcemy oszacować tłoczność miejsca.